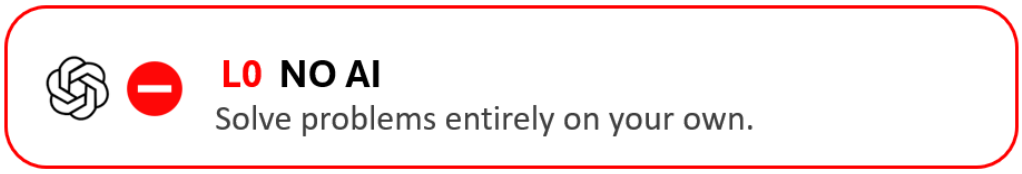
W1 PRACTICE

PART 1 – EXPLORATION

*What do you need to **be able to do** for next week?*

- Identify the **type** of variable
- Manipulate **nullable** and **non-nullable** variables.
- Differentiate between **final** and **const**.
- Manipulate **strings, lists, and maps.**
- Use **loops** and **conditions** to control flow.
- Define and call functions with positional and **named arguments**, understand **arrow syntax**



How to run Dart code?

You can write you code on VS Code, or using this online editor

- Install Dart and Flutter SDK
- Online Dart Compiler

Resources for this research

To help you complete this handout, use the following resources:

- ✓ Variables
- ✓ Null Safety
- ✓ Built-in types
- ✓ Lists
- ✓ Loops
- ✓ Conditions
- ✓ Functions

EX 1 - Type Inference

Built-in types

EXPLAIN : Explain how Dart infers the type of a variable.

Dart automatically infers the type of variable when a variable is assigned a value bcus it can recognize the data type of literals.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a int variable using var or final and let Dart infer its type
var num = 1;
// Define a variable with an explicit String type
String name = 'tyty';
```

EX 2 - Nullable and Non-Nullable Variables

Variables Null Safety

EXPLAIN : Explain nullable vs non-nullable variables.

Nullable variables are variables that can be null and can default to null.
Non-nullable variables are variables that dart doesn't set initial values to, meaning a value need to be assigned/initialize first.

EXPLAIN : When is it useful to have nullable variables?

It is useful when we want to be able to have optional input data.
Example: profile bio.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a nullable integer variable and assign it a null value
int? x = null;
// Declare a non-nullable integer variable and assign it a value
int y = 1;
// Assign a new value to the nullable variable
int? num;
num = 5;
```

EX 3 - Final and const

EXPLAIN : Describe the difference between final and const.

A final object cannot be changed but its field can still be change, it is set at runtime. A const object and its field cannot be change, it is set at compile-time, meaning its data is read is set when set when the code is built.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a final variable and assign it the current date and time
final curtime = DateTime.now();
// Can you declare this variable as const? Why?
No, bcus const have a frozen value meaning it is always constant is isn't
changeable but the value of DateTime.now() function always change.
// Declare a const variable with a integer value
const c = 2;
// Can you reassign the value of this final variable? Why?
No, bcus const and final variables cannot be reassigned after initialization.
```

EX 4 - Strings, Lists and Maps

Lists Built-in types

CODE : Complete the bellow code to illustrate the concepts:

Strings:

```
// Declare two strings: firstName and lastName and an integer:age
String firstName = 'Chey';
String lastName = 'Naryvety';
int age = 19;
// Concatenate the 2 strings and the age
var info = '$firstName $lastName is $age years old.'
// Print result
print(info);
```

Lists:

```
// Create a list of integers
var list = [1,2,3];
// Add a number to the list
list.add(4);
// Remove a number from the list
list.remove(3);
// Insert a number at a specific index in the list
list.insert(2,8);
// Iterate over the list and print each number
for (int i=0;i<list.length;i++) {
        print(list[i]);
}
```

Maps:

```
// Create a map with String keys and integer values
var studentScore = {
      'Seth': 95,
      'Omra': 92,
      'Ty': 100,
}
// Add a new key-value pair to the map
studentScore['Panha'] = 90;
```

```
// Remove a key-value pair from the map
studentScore.remove('Omra');
// Iterate over the map and print each key-value pair
studentScore. forEach((key, value) => print("Name: $key, Score: $value"));
```

EXPLAIN : When should I use a Map instead of a List?

Use Map when data uses unique identifier (keys) for fast lookups like using id instead of a ordered index.

EXPLAIN : When should I use a Set instead of a List?

Use set when data must store unique values and shouldn't have repeating values for a fast lookup.

EX 5 - Loops and Conditions

Loops Conditions

CODE : Complete the bellow code to illustrate the concepts:

```
// Use a for-loop to print numbers from 1 to 5
for (i=0,i<5,i++){
      print(i+1);
}
// Use a while-loop to print numbers while a condition is true
var i = 0;
while (i<5) {
      print(i);
      i++;
}
// Use an if-else statement to check if a number is even or odd
var x = 10
if (x%2==0) {
      print('even');
} else {
      print('odd');
}
```

EX 6 - Functions

Functions

EXPLAIN : Compare positional and named function arguments

Positional Arguments:
arguments/parameters are wrapped in []
arguments must match the exact order of the parameters defined
cannot be null
Named Arguments:
arguments/parameters are wrapped in {}
must type the parameter name but arguments can be in any order
can be null

EXPLAIN : Explain when and how to use arrow syntax for functions?

A shorthand => (*arrow*) syntax is useful for functions that contain a single statement/one line of code. It acts as/replace { return ... ; } by automatically returning the value.

CODE : Complete the bellow code to illustrate the concepts:

Defining and Invoking a Function:

```
// Define a function that takes two integers and returns their sum
var add = (int a, int b) => a + b;
// Call the function and print the result
int result = add(6, 7);
print('total is $result');
```

Positional vs Named Arguments:

```
// Define a function that uses positional arguments
void info(String firstName, String lastName, int age, [String? middleName]) {
      var fullName = firstName + " " + lastName;
      if (middleName != null) {
          fullName = firstName + " " + middleName + " " + lastName;
      }
print('$fullName is $age years old.'); }
// Define another function that uses named arguments with the required
keyword (ex: getArea with rectangle arguments)
var getArea = ({required double width, required double height}) {
      return width * height;
};
// Call both functions with appropriate arguments
info('Nob', 'Johnson', 20);
info ('Salim', 'Both', 21, 'Virak');
var area = getArea(height: 6.9, width: 6.7);
print('Rectangle area is $area')
```

EXPLAIN : Can positional argument be omitted? Show an example

Yes, if it is wrapped in [] meaning it can be optional. Regular positional arguments cannot be omitted. Example is show in exercise above.

EXPLAIN : Can named argument be omitted? Show an example

Yes, by default it can be omitted because they are optional but if we use the keyword required like in the example above then it cannot be omitted.

CODE : Complete the bellow code to illustrate the concepts:

Arrow Syntax:

```
// Define a function using arrow syntax that squares a number
var square = (int x) => x * x;
// Call the arrow function and print the result
print(square(5));
//or
var number = square(5);
print(number)
```